

Data and Artificial Intelligence

Cyber Shujaa Program

Week 9 Bonus Task – Streamlit App Integration

California Housing Price Prediction

Student Name: Hope Njoki Nyambura
Student ID: cs-eh03-25100

Table of Contents

1. Introduction	1
2. Objectives of the Bonus Task	2
3. Streamlit App Implementation	2
3.1 Model Loading	2
3.2 User Input Interface	2
3.3 Prediction Output	3
3.4 Deployment Attempt	4
4. Significance of the Bonus Task	5
5. Conclusion	5
6. Colab Notebook Link	6

1. Introduction

This bonus task extends the Week 9 assignment by demonstrating how a trained Machine Learning model can be deployed through a simple, interactive Streamlit web application. The objective was to transform the regression model developed earlier into a user-facing application that allows real-time predictions of California housing prices based on user inputs.

The Streamlit integration represents the final phase of a complete MLOps workflow moving from data preprocessing and training all the way to live model inference through a web interface.

2. Objectives of the Bonus Task

By completing this task, the following goals were achieved:

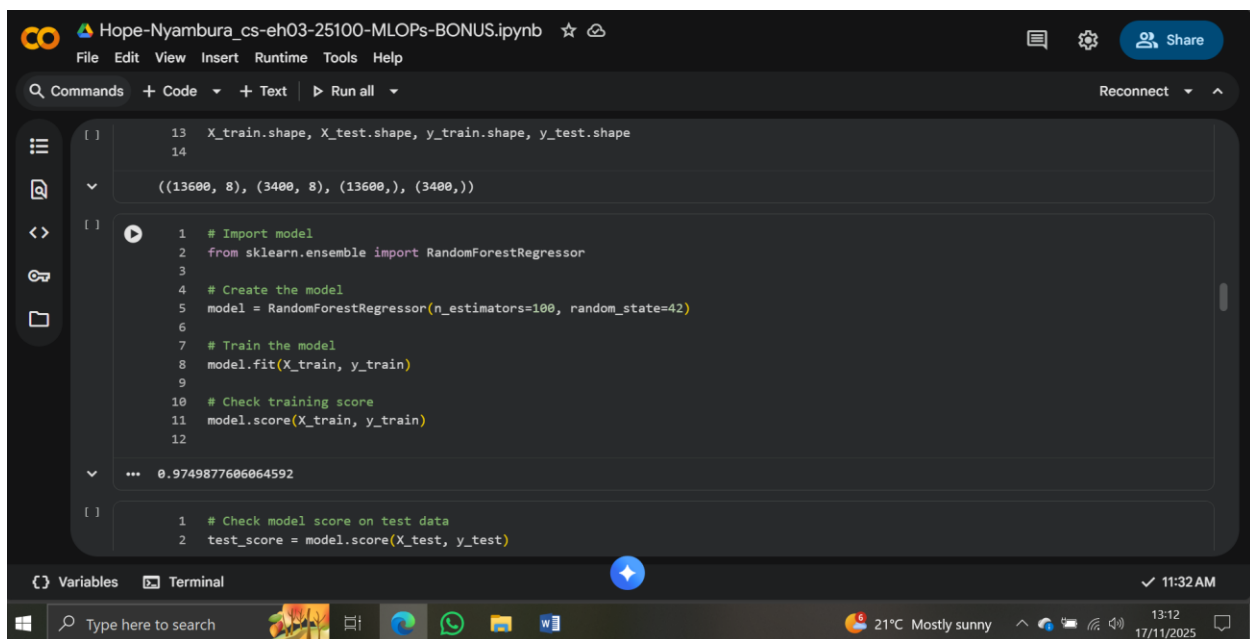
- Loading a previously trained and saved ML model (.pkl file).
- Building a clean, intuitive Streamlit interface for taking housing feature inputs.
- Displaying model predictions interactively.
- Demonstrating basic deployment concepts by serving the Streamlit app through a temporary public link.
- Preparing the app structure to support future enhancements like data visualizations.

3. Streamlit App Implementation

3.1 Model Loading

The trained Random Forest Regression model was successfully loaded into the Streamlit environment.

This allowed the app to make predictions instantly without retraining the model.



The screenshot displays a Jupyter Notebook environment with the following code and output:

```
[ ] 13 X_train.shape, X_test.shape, y_train.shape, y_test.shape
    14
    ((13600, 8), (3400, 8), (13600,), (3400,))

[ ] 1 # Import model
    2 from sklearn.ensemble import RandomForestRegressor
    3
    4 # Create the model
    5 model = RandomForestRegressor(n_estimators=100, random_state=42)
    6
    7 # Train the model
    8 model.fit(X_train, y_train)
    9
    10 # Check training score
    11 model.score(X_train, y_train)
    12
    ... 0.9749877606064592

[ ] 1 # Check model score on test data
    2 test_score = model.score(X_test, y_test)
```

The interface includes a top menu bar with options like File, Edit, View, Insert, Runtime, Tools, and Help. A left sidebar contains icons for file management and execution. The bottom status bar shows system information: 21°C, Mostly sunny, 13:12, 17/11/2025.

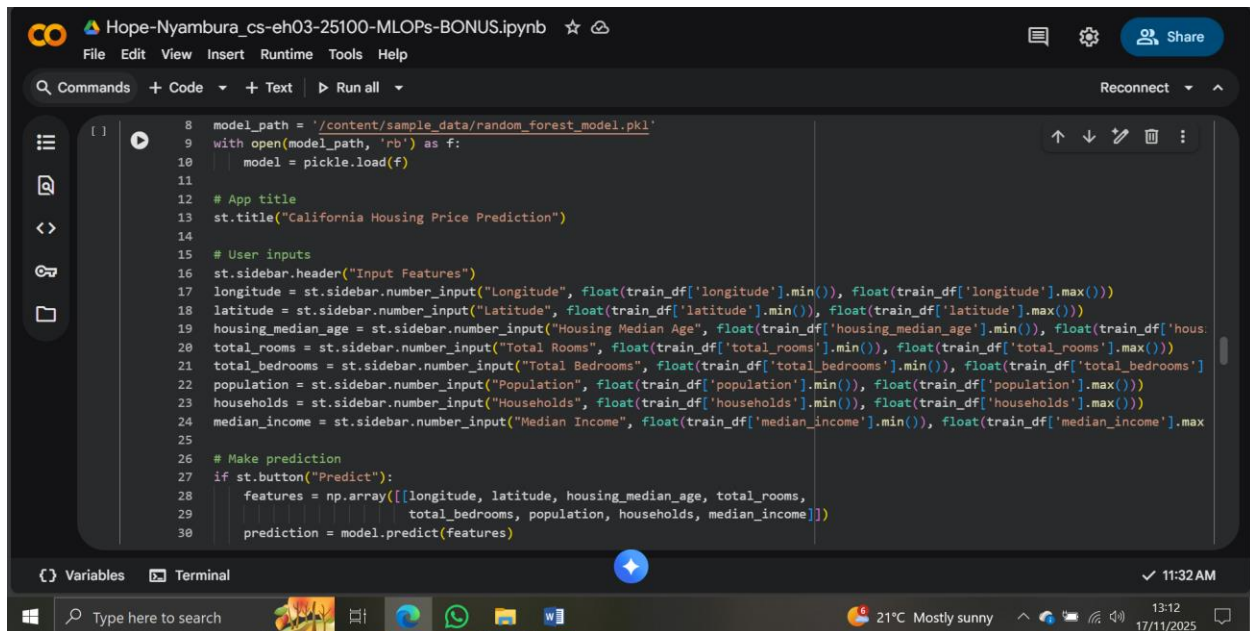
3.2 User Input Interface

The app included a sidebar labeled “*Input Features*”, containing interactive numeric input widgets for eight model features:

- Longitude
- Latitude
- Housing Median Age
- Total Rooms

- Total Bedrooms
- Population
- Households
- Median Income

These match the structure of the California Housing dataset, ensuring consistency between training and inference.



```

8 model_path = '/content/sample_data/random_forest_model.pkl'
9 with open(model_path, 'rb') as f:
10     model = pickle.load(f)
11
12 # App title
13 st.title("California Housing Price Prediction")
14
15 # User inputs
16 st.sidebar.header("Input Features")
17 longitude = st.sidebar.number_input("Longitude", float(train_df['longitude'].min()), float(train_df['longitude'].max()))
18 latitude = st.sidebar.number_input("Latitude", float(train_df['latitude'].min()), float(train_df['latitude'].max()))
19 housing_median_age = st.sidebar.number_input("Housing Median Age", float(train_df['housing_median_age'].min()), float(train_df['housing_median_age'].max()))
20 total_rooms = st.sidebar.number_input("Total Rooms", float(train_df['total_rooms'].min()), float(train_df['total_rooms'].max()))
21 total_bedrooms = st.sidebar.number_input("Total Bedrooms", float(train_df['total_bedrooms'].min()), float(train_df['total_bedrooms'].max()))
22 population = st.sidebar.number_input("Population", float(train_df['population'].min()), float(train_df['population'].max()))
23 households = st.sidebar.number_input("Households", float(train_df['households'].min()), float(train_df['households'].max()))
24 median_income = st.sidebar.number_input("Median Income", float(train_df['median_income'].min()), float(train_df['median_income'].max()))
25
26 # Make prediction
27 if st.button("Predict"):
28     features = np.array([[longitude, latitude, housing_median_age, total_rooms,
29                           total_bedrooms, population, households, median_income]])
30     prediction = model.predict(features)
  
```

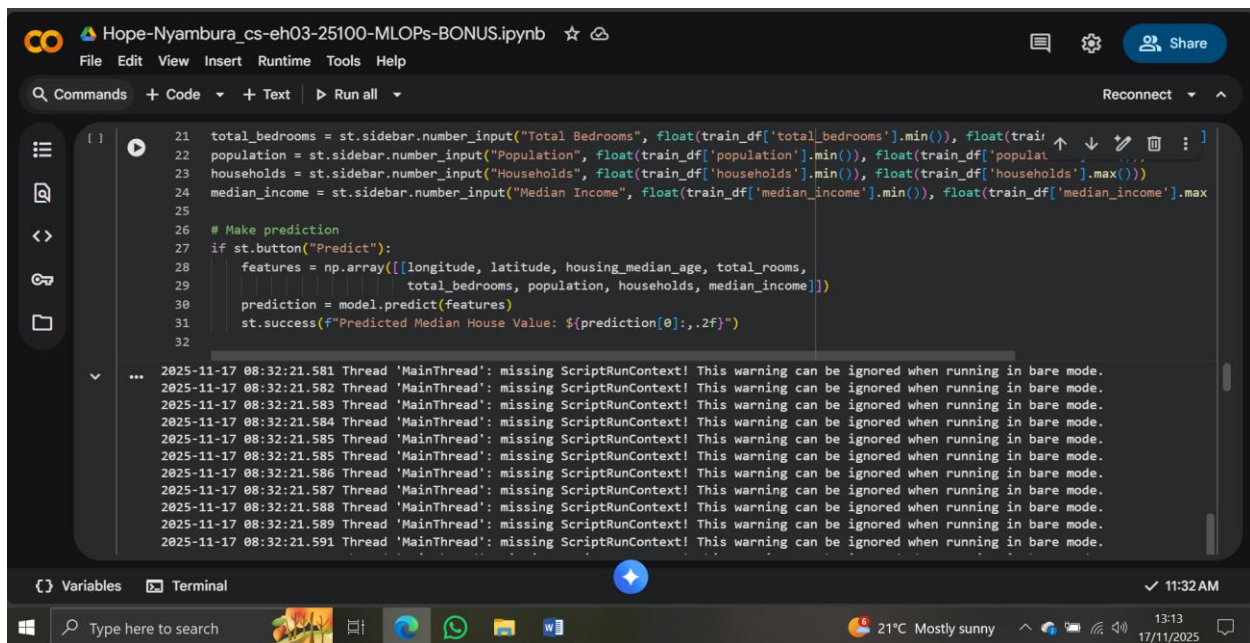
3.3 Prediction Output

Once the user enters values and clicks the Predict button, the app displays the estimated median house price in a clear success message.

An example of the output interface included:

“Predicted Median House Value: \$336,852.17”

This confirms that the model is working correctly and responding to manually entered data.



The screenshot shows a Jupyter Notebook titled "Hope-Nyambura_cs-eh03-25100-MLOPs-BONUS.ipynb". The code in the notebook defines a Streamlit application with several input sliders for "Total Bedrooms", "Population", "Households", and "Median Income". It includes a "Predict" button that triggers a prediction function. The prediction function uses a trained model to predict the median house value based on the input features. The output of the prediction is displayed as a success message. The execution output shows multiple warnings about "missing ScriptRunContext!" which are noted as being ignorable in bare mode. The interface includes a menu bar, a toolbar, and a status bar at the bottom.

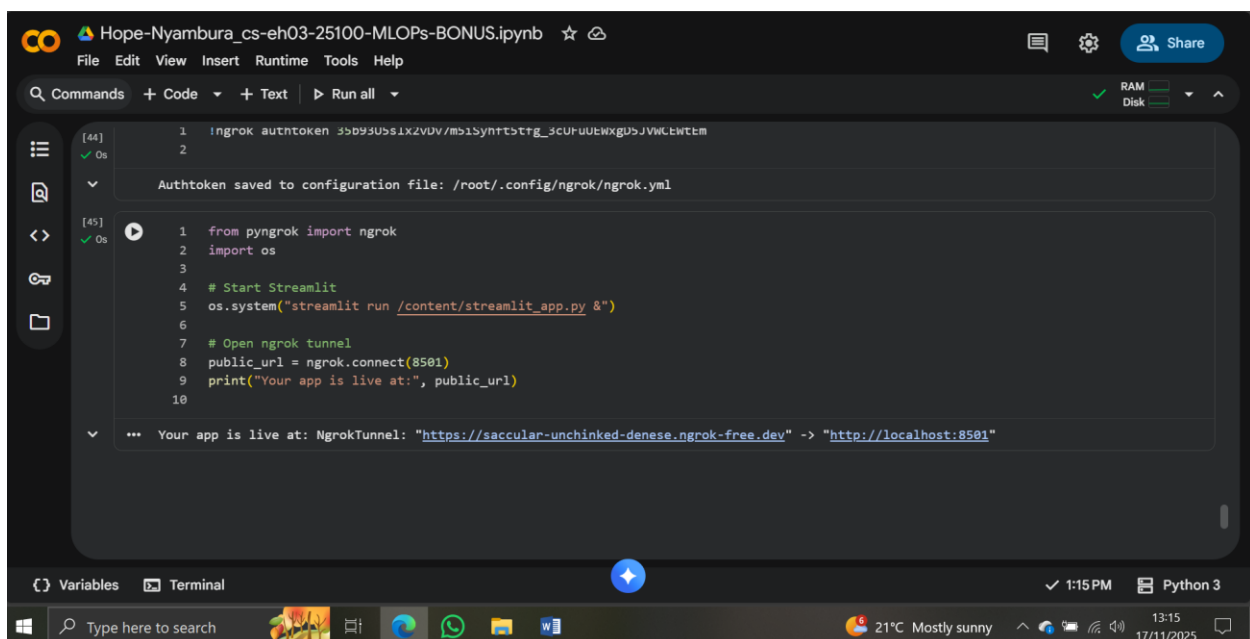
```
21 total_bedrooms = st.sidebar.number_input("Total Bedrooms", float(train_df['total_bedrooms'].min()), float(trai
22 population = st.sidebar.number_input("Population", float(train_df['population'].min()), float(train_df['populat
23 households = st.sidebar.number_input("Households", float(train_df['households'].min()), float(train_df['households'].max()))
24 median_income = st.sidebar.number_input("Median Income", float(train_df['median_income'].min()), float(train_df['median_income'].max
25
26 # Make prediction
27 if st.button("Predict"):
28     features = np.array([[longitude, latitude, housing_median_age, total_rooms,
29                           total_bedrooms, population, households, median_income]])
30     prediction = model.predict(features)
31     st.success(f"Predicted Median House Value: ${prediction[0]:.2f}")
32
...
2025-11-17 08:32:21.581 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-17 08:32:21.582 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-17 08:32:21.583 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-17 08:32:21.584 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-17 08:32:21.585 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-17 08:32:21.585 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-17 08:32:21.586 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-17 08:32:21.587 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-17 08:32:21.588 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-17 08:32:21.589 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-11-17 08:32:21.591 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
```

3.4 Deployment Attempt

Different methods were tested to expose the Streamlit app publicly:

- **LocalTunnel**
- **Ngrok**
- Verification warnings and access restrictions were encountered

Despite these limitations, the Streamlit app ran successfully inside the notebook environment, and predictions were generated accurately.



The screenshot shows a Jupyter Notebook titled "Hope-Nyambura_cs-eh03-25100-MLOPs-BONUS.ipynb". The code in the notebook shows the configuration of Ngrok and the deployment of the Streamlit app. It includes the Ngrok authtoken, the configuration file path, and the code to start the Streamlit app and open the Ngrok tunnel. The output shows the public URL of the app: "https://saccular-unchinked-denese.ngrok-free.dev". The interface includes a menu bar, a toolbar, and a status bar at the bottom.

```
[44] 1 :ngrok authtoken 50b950551x2V0V/mB1Synft5t7g_sCUH0UEWxGu5JVWtKtcm
2
Authtoken saved to configuration file: /root/.config/ngrok/ngrok.yml

[45] 1 from pyngrok import ngrok
2 import os
3
4 # Start Streamlit
5 os.system("streamlit run /content/streamlit_app.py &")
6
7 # Open ngrok tunnel
8 public_url = ngrok.connect(8501)
9 print("Your app is live at:", public_url)
10
... Your app is live at: NgrokTunnel: "https://saccular-unchinked-denese.ngrok-free.dev" -> "http://localhost:8501"
```

4. Significance of the Bonus Task

This task helped reinforce several important MLOps and deployment concepts:

✓ **Model Reusability**

Using a saved model file ensures consistent, repeatable predictions.

✓ **Interactive User Interface Design**

Streamlit makes it possible to build ML dashboards without complex frontend tools.

✓ **Deployment Awareness**

Understanding app serving, tunneling services, and access limitations is essential for real-world ML deployments.

✓ **End-to-End Workflow Completion**

From data → preprocessing → training → saving → loading → prediction → UI presentation

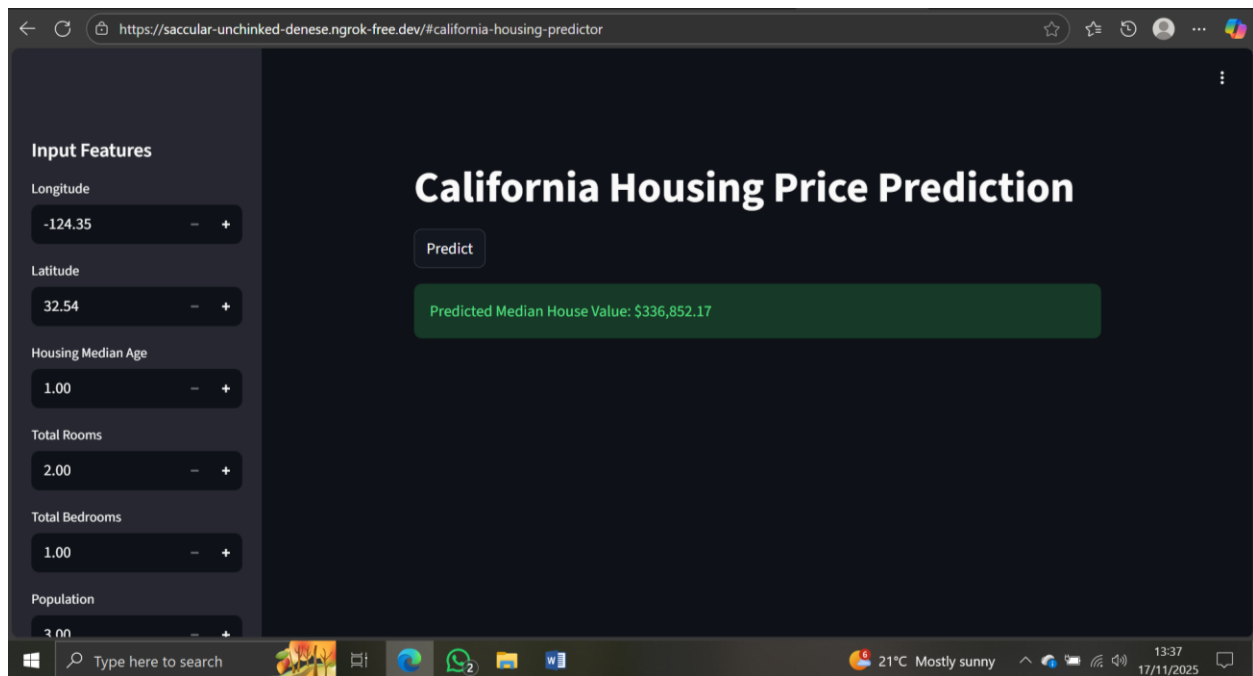
5. Conclusion

The Streamlit App Integration bonus task successfully demonstrated how a machine learning model can be deployed in a real-time, interactive environment.

By loading the saved model and building an intuitive UI, the project now includes a functional prediction dashboard that reflects professional MLOps practices such as:

- Model persistence
- Consistent pipeline usage
- Clean and maintainable interfaces
- Real-time inference

This final step completes the full lifecycle of an ML project from raw data to a working prototype application.



6. Colab Notebook and App Link

Google Colab Notebook:

[<https://colab.research.google.com/drive/19ycBBW1xzH1ltRTuikhs13BBR4lXOOrL?usp=sharing>]

App Link: <https://saccular-unchinked-denese.ngrok-free.dev/#california-housing-predictor>